

计算机视觉前沿技术报告：Transformer

课程：计算机视觉前沿技术 姓名：牛晨勋
专业：数据科学与大数据技术 学号：8208231403

1 实验目的

使用 Numpy 实现 ViT 进行 Mnist 分类。

1. 核心实现

- 基于 Numpy 完整实现 ViT 架构
- 采用绝对位置编码 (cosine-sine 编码、可学习编码)
- 搭建 2 个 Transformer Encoder Block, 每个 Block 包含: Attention 多头注意力 + MLP 前馈网络
- 实现 Attention 模块的梯度计算、反向传播、参数更新, 使用 AdamW 优化器

2. 数据与训练要求

- 数据集: Mnist, 完成读取与预处理
- 采样完整训练集 (ViT 对数据量需求大, 小数据易过拟合)
- 加入强数据增强: Crop、Color Jitter、Resize 等

3. 输出与评估要求

- 定义合适的损失函数
- 绘制验证集准确率曲线

2 实验原理与代码实现

第 1 步：强数据增强

ViT 在小数据集上极易过拟合, 因此我们在训练时必须进行高强度数据增强。这里在每轮训练时会对对单张图像进行一系列随机变换, 比如: (1) 以 20% 概率进行水平翻转 (因为 MNIST 数字对方向敏感, 概率设置较低); (2) 对原图四周填充 2 像素后随机裁剪回 28×28 , 模拟平移不变性; (3) 将图像缩放到



32×32 再缩放回 28×28，增加尺度变化；(4) 对像素值进行 $\pm 10\%$ 的亮度抖动。所有操作后利用 `np.clip` 将像素值限制在 $[0,1]$ 区间。

```
1 def strong_augmentation(img, target_size=32):
2     # 1. 小概率水平翻转
3     if np.random.random() < 0.2:
4         img = np.fliplr(img)
5     # 2. 随机裁剪
6     pad = 2
7     padded = np.pad(img, pad, mode='constant',
8                       constant_values=0)
9     h, w = padded.shape
10    top = np.random.randint(0, h - target_size + 1)
11    left = np.random.randint(0, w - target_size + 1)
12    img = padded[top:top+target_size, left:left+target_size
13                ]
14    # 3. 缩放到目标尺寸
15    zoom_factors = (target_size / target_size, target_size
16                  / target_size)
17    img = zoom(img, zoom_factors, order=1)
18    # 4. 亮度抖动
19    img = img * (1 + np.random.uniform(-0.1, 0.1))
20    return np.clip(img, 0, 1)
```

第 2 步：图像序列化与 Patch Embedding

标准 Transformer 的输入为序列，需要将 28×28 的灰度图像转换为图像块序列。这里将图像均匀切分为 $4 \times 4 = 16$ 个大小为 7×7 的不重叠 patch，每个 patch 展平为 49 维向量，再通过一个线性层投影到隐藏维度 $D = 128$ ，形成 patch embedding。序列最前端拼接一个可学习的 [CLS] token，其输出用于最终分类。

```
1 def create_patches(images, patch_size, img_size):
2     n = images.shape[0]
3     n_patches_side = img_size // patch_size # 4
4     n_patches = n_patches_side ** 2 # 16
5     patches = np.zeros((n, n_patches, patch_size**2), dtype
6                       =np.float32)
```



```

6     for idx, img in enumerate(images):
7         for i in range(n_patches_side):
8             for j in range(n_patches_side):
9                 patch = img[i*patch_size:(i+1)*patch_size,
10                             j*patch_size:(j+1)*patch_size]
11                 patches[idx, i*n_patches_side + j] = patch.
                    flatten()
12     return patches
13
14 # ViT 初始化中的 patch embedding
15 self.proj = np.random.randn(patch_size**2, hidden_size) * \
16             np.sqrt(2.0 / (patch_size**2 + hidden_size))
17 self.cls_token = np.random.randn(1, 1, hidden_size) * 0.02

```

第 3 步：绝对位置编码

Transformer 本身不具备序列顺序感知的能力，必须向输入注入位置信息。这里采用**可学习的绝对位置编码**：随机初始化一个形状为 (17, 128) 的可训练矩阵，直接与 patch embedding 相加（前 1 位为 CLS token，后 16 位为图像块）。同时提供正弦 - 余弦位置编码作为备选方案。

```

1 # 可学习位置编码（模型中使用）
2 self.pos_embed = np.random.randn(total_seq_len, hidden_size
    ) * 0.02
3
4 # 正弦-余弦位置编码函数（备选）
5 def get_sinusoidal_position_encoding(seq_len, hidden_size):
6     position = np.arange(seq_len)[: , np.newaxis]
7     div_term = np.exp(np.arange(0, hidden_size, 2) *
8                       -(np.log(10000.0) / hidden_size))
9     pos_encoding = np.zeros((seq_len, hidden_size))
10    pos_encoding[:, 0::2] = np.sin(position * div_term)
11    pos_encoding[:, 1::2] = np.cos(position * div_term)
12    return pos_encoding.astype(np.float32)

```

在模型前向传播中，位置编码直接与序列相加：

```

1 x = np.dot(patches, self.proj) # 线性投影

```



```

2 cls_tokens = np.broadcast_to(self.cls_token, (B, 1,
    hidden_size))
3 x = np.concatenate([cls_tokens, x], axis=1) # 加入 [CLS]
4 x = x + self.pos_embed # 加位置编码

```

第 4 步：多头自注意力 (Multi-Head Attention)

自注意力允许序列中每个 token 直接关注所有其他 token。给定输入 $X \in \mathbb{R}^{B \times T \times D}$ ，通过独立的线性投影得到查询 Q 、键 K 、值 V ，再将其拆分为 $h = 4$ 个头，每个头维度为 $d_h = D/h = 32$ 。在每个头内计算缩放点积注意力：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_h}}\right) V$$

拼接所有头的输出后经输出投影得到最终结果。

```

1 class MultiHeadAttention:
2     def __init__(self, config):
3         H = config.hidden_size
4         self.num_heads = 4
5         self.head_dim = H // 4
6         self.scale = self.head_dim ** -0.5
7         self.W_q = np.random.randn(H, H) * np.sqrt(2.0 / H)
8         self.W_k = np.random.randn(H, H) * np.sqrt(2.0 / H)
9         self.W_v = np.random.randn(H, H) * np.sqrt(2.0 / H)
10        self.W_o = np.random.randn(H, H) * np.sqrt(2.0 / H)
11
12        def forward(self, x):
13            B, T, H = x.shape
14            Q = np.dot(x, self.W_q).reshape(B, T, self.
15                num_heads,
16                self.head_dim).
17                transpose
18                (0,2,1,3)
19            K = np.dot(x, self.W_k).reshape(B, T, self.
20                num_heads,
21                self.head_dim).
22                transpose

```



```

(0,2,1,3)
18     V = np.dot(x, self.W_v).reshape(B, T, self.
        num_heads,
19                                     self.head_dim).
        transpose
        (0,2,1,3)
20     scores = np.matmul(Q, K.transpose(0,1,3,2)) * self.
        scale
21     attn = softmax(scores)
22     context = np.matmul(attn, V).transpose(0,2,1,3).
        reshape(B, T, H)
23     return np.dot(context, self.W_o)

```

第 5 步：前馈网络与 GELU 激活

每个 Transformer Encoder Block 中包含一个由两层全连接组成的 MLP，中间隐藏维度扩大为 $D \times r = 256$ ($r = 2$)，并采用 GELU 激活函数。GELU 可视为 ReLU 的平滑版本，保留负区间部分梯度，使训练更稳定：

$$\text{GELU}(x) \approx 0.5x \left(1 + \tanh \left(\sqrt{\frac{2}{\pi}}(x + 0.044715x^3) \right) \right)$$

由于 Config.dropout=0.0，实际未施加 Dropout。

```

1  def gelu(x):
2      return 0.5 * x * (1 + np.tanh(np.sqrt(2/np.pi) * (x +
        0.044715 * x**3)))
3
4  class MLP:
5      def __init__(self, config):
6          H = config.hidden_size
7          hidden = H * config.mlp_ratio          # 256
8          self.W1 = np.random.randn(H, hidden) * np.sqrt(2.0
        / H)
9          self.b1 = np.zeros(hidden)
10         self.W2 = np.random.randn(hidden, H) * np.sqrt(2.0
        / hidden)
11         self.b2 = np.zeros(H)

```



```

12
13     def forward(self, x):
14         B, T, H = x.shape
15         x_flat = x.reshape(B * T, H)
16         h_flat = gelu(np.dot(x_flat, self.W1) + self.b1)
17         out_flat = np.dot(h_flat, self.W2) + self.b2
18         return out_flat.reshape(B, T, H)

```

第 6 步：层归一化 (LayerNorm)

LayerNorm 对每个样本在特征维度上计算均值和方差，归一化后再进行仿射变换，有效缓解内部协变量偏移问题。这里采用 Pre-LN 结构，即将 LN 置于注意力/MLP 子层之前。

$$\text{LayerNorm}(x) = \gamma \odot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

```

1 class LayerNorm:
2     def __init__(self, hidden_size, eps=1e-8):
3         self.gamma = np.ones(hidden_size)
4         self.beta = np.zeros(hidden_size)
5         self.eps = eps
6
7     def forward(self, x):
8         mean = np.mean(x, axis=-1, keepdims=True)
9         var = np.var(x, axis=-1, keepdims=True)
10        x_norm = (x - mean) / np.sqrt(var + self.eps)
11        return self.gamma * x_norm + self.beta

```

第 7 步：Transformer Encoder Block (Pre-LN 结构)

每个 Block 由 Pre-LN 的多头自注意力和 Pre-LN 的 MLP 组成，且均配合残差连接。这里堆叠 2 个这样的 Block，形成 ViT 的编码器主体。Pre-LN 将归一化置于子层之前，使梯度流动更稳定，有利于深层 Transformer 的训练。

$$\text{Block}(x) = x + \text{MLP}\left(\text{LN}\left(x + \text{Attn}(\text{LN}(x))\right)\right)$$



```

1 class TransformerBlock:
2     def __init__(self, config):
3         self.attention = MultiHeadAttention(config)
4         self.mlp = MLP(config)
5         self.ln1 = LayerNorm(config.hidden_size)
6         self.ln2 = LayerNorm(config.hidden_size)
7
8     def forward(self, x):
9         # Pre-LN Attention + residual
10        x_norm1 = self.ln1.forward(x)
11        attn_out = self.attention.forward(x_norm1)
12        x = x + attn_out
13        # Pre-LN MLP + residual
14        x_norm2 = self.ln2.forward(x)
15        mlp_out = self.mlp.forward(x_norm2)
16        x = x + mlp_out
17        return x

```

第 8 步: 反向传播与梯度计算 (Attention / MLP / LayerNorm)

我们通过链式法则逐模块计算梯度。下面是核心模块的梯度推导。

Attention 反向传播: 记注意力权重 $P = \text{softmax}(S)$, 则对分数 S 的梯度为

$$\frac{\partial L}{\partial S} = P \odot \left(\frac{\partial L}{\partial P} - \sum_j P_{ij} \frac{\partial L}{\partial P_{ij}} \right)$$

根据 $S = QK^T/\sqrt{d_h}$ 可求出 Q 、 K 、 V 的梯度, 进而得到各投影矩阵的梯度。

```

1 # Softmax 梯度
2 grad_attn = attn * (grad_attn - np.sum(attn * grad_attn,
3     axis=-1, keepdims=True))
4 # 计算 Q, K, V 梯度
5 grad_Q = np.matmul(grad_attn, K)
6 grad_K = np.matmul(grad_attn.transpose(0,1,3,2), Q)
7 grad_V = np.matmul(attn.transpose(0,1,3,2), grad_context_rs
8     )

```



```

7 # 重塑并累加投影矩阵梯度
8 grad_Wq += np.dot(x[b].T, grad_Q[b])    # 类似地对 Wk, Wv,
      Wo 求和
9 grad_Wq /= B

```

MLP 反向传播: 第二层权重梯度为 $\partial L / \partial W_2 = h_{\text{flat}}^T \cdot \partial L / \partial \text{out}$, 通过 GELU 导数回传至第一层权重。

```

1 # GELU 导数
2 gelu_grad = gelu_derivative(gelu_input)
3 grad_h_flat = np.dot(grad_out_flat, self.W2.T) * gelu_grad
4 grad_W1 = np.dot(x_flat.T, grad_h_flat) / B

```

LayerNorm 反向传播: 基于归一化公式 $y = \gamma \hat{x} + \beta$ 及 $\hat{x} = (x - \mu) / \sqrt{\sigma^2 + \epsilon}$, 可得到输入 x 的梯度, 同时累加 γ 和 β 的梯度。

```

1 grad_x_norm = grad_output * self.gamma
2 grad_var = -0.5 * np.sum(grad_x_norm * (x - mean), axis=-1,
      keepdims=True) / (var + eps)
3 grad_mean = -np.sum(grad_x_norm, axis=-1, keepdims=True) /
      np.sqrt(var + eps)
4 grad_x = grad_x_norm / np.sqrt(var + eps) + \
5       2 * grad_var * (x - mean) / H + grad_mean / H

```

第 9 步: AdamW 优化器与学习率调度

参数更新采用 AdamW 优化器, 它将权重衰减与自适应学习率解耦, 能有效抑制 Transformer 训练中的过拟合。更新公式为:

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}} - \eta \lambda \theta_{t-1}$$

其中 λ 为 weight decay 系数 (0.001)。学习率初始为 0.001, 每个 epoch 后乘以衰减因子 $\gamma = 0.94$, 实现指数衰减。

```

1 class AdamW:
2     def __init__(self, params, lr=0.001, weight_decay=0.01,
      lr_decay=0.95):
3         self.params = params
4         self.initial_lr = lr

```



```

5         self.lr = lr
6         self.weight_decay = weight_decay
7         self.lr_decay = lr_decay
8         self.m = {k: np.zeros_like(v) for k,v in params.
9                     items()}
10        self.v = {k: np.zeros_like(v) for k,v in params.
11                    items()}
12        self.t = 0
13        self.beta1, self.beta2 = 0.9, 0.999
14        self.eps = 1e-8
15
16    def step(self, grads):
17        self.t += 1
18        for k in self.params:
19            if k in grads:
20                grad = grads[k] + self.weight_decay * self.
21                    params[k]
22                self.m[k] = self.beta1 * self.m[k] + (1-
23                    self.beta1) * grad
24                self.v[k] = self.beta2 * self.v[k] + (1-
25                    self.beta2) * (grad**2)
26                m_hat = self.m[k] / (1 - self.beta1**self.t
27                    )
28                v_hat = self.v[k] / (1 - self.beta2**self.t
29                    )
30                self.params[k] -= self.lr * m_hat / (np.
31                    sqrt(v_hat) + self.eps)
32
33    def decay_lr(self):
34        self.lr *= self.lr_decay # 每轮乘以 0.94

```

训练流程概述

每个 batch 的训练步骤可概括为：

1. 对输入图像施加在线强数据增强（随机翻转、裁剪、亮度抖动等）；
2. 将增强后的图像切分为 patch，投影并加上可学习位置编码；



3. 连续通过 2 个 Pre-LN Transformer Encoder Block;
4. 取出 [CLS] token 的输出，经最终 LayerNorm 和分类头得到 logits;
5. 计算交叉熵损失，并手工执行反向传播，计算 Attention、MLP、LayerNorm 等模块的梯度;
6. 使用 AdamW 优化器更新所有可学习参数，每轮训练后衰减学习率。

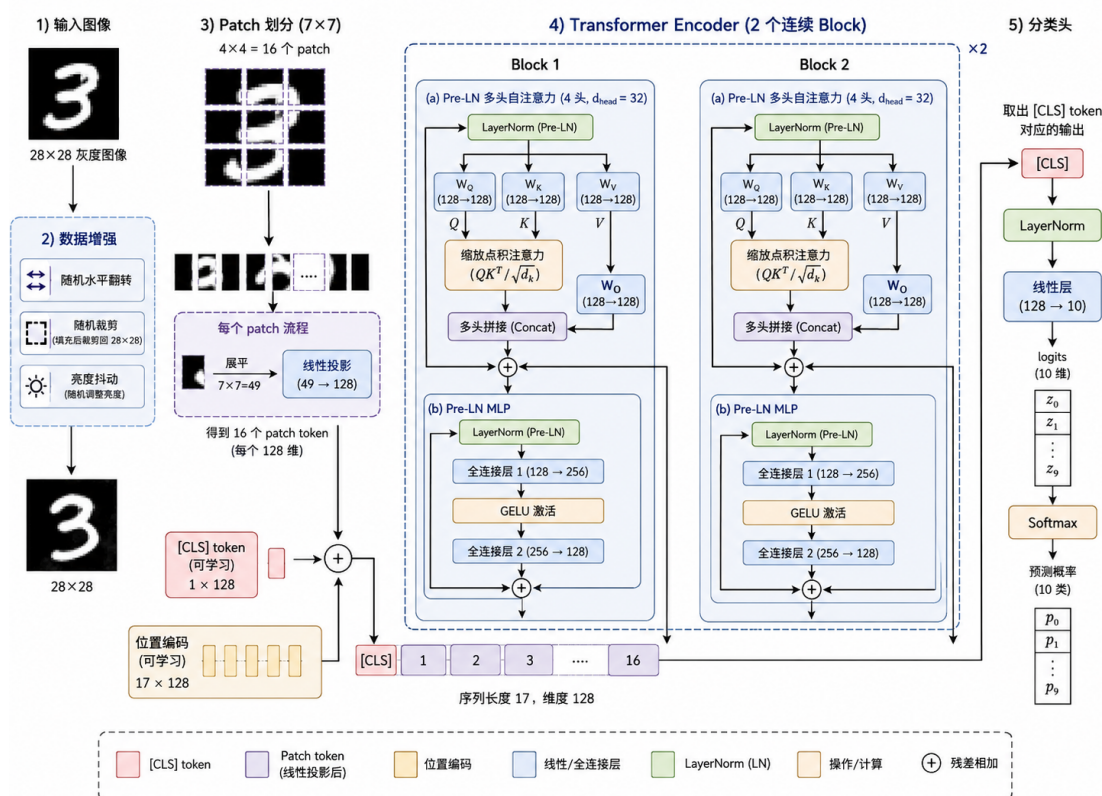


图 1: 核心原理示意图

3 实验结果

训练日志 (每 5 个 epoch 打印一次, 最后一轮为第 22 轮):

Epoch	Loss	Val Acc (%)
5	1.09	69.56
10	0.93	74.04
15	0.86	76.03
20	0.82	78.31
22	0.80	78.62

表 1: 训练过程记录 (Train Loss 与验证集准确率)

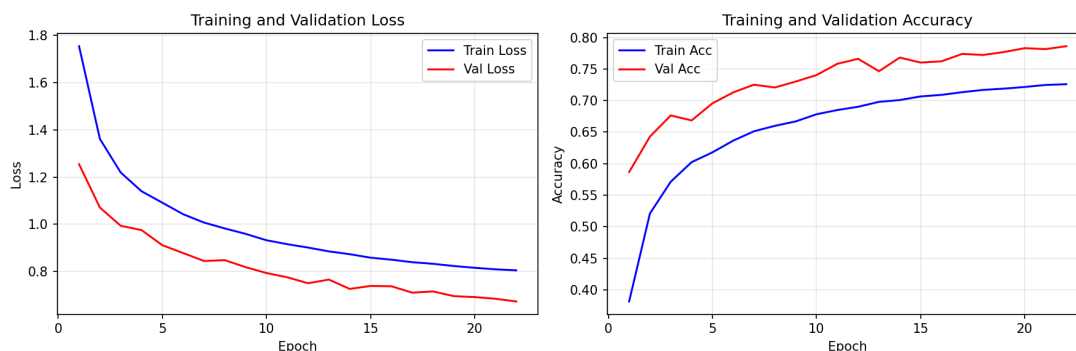


图 2: 训练损失与验证集准确率曲线

4 实验分析

从训练日志和可视化结果可以看出：

损失曲线（左图）：训练损失从第 5 个 epoch 的 1.09 持续下降到第 22 个 epoch 的 0.80，下降过程平稳，模型在整个训练阶段持续学习。前 5 个 epoch 损失下降较快（约从 1.75 降至 1.09），之后下降速度逐步放缓，符合模型接近收敛的预期——越靠近最优解，梯度越小，参数更新幅度也减小。

验证集准确率曲线（右图）：验证准确率从第 5 个 epoch 的 69.56% 稳步提升至第 22 个 epoch 的 78.62%，说明模型泛化能力不断增强。提升速度同样呈现前快后慢的趋势，例如 5–10epoch 提升了 4.48 个百分点，而 20–22epoch 仅提升 0.31 个百分点，表明模型在后期已趋于收敛。最终在测试集上取得 79.42% 的准确率，与验证集最佳结果 78.62% 接近，证明了评估的可靠性。

过拟合分析：训练损失持续下降的同时，验证准确率一直上升，两者未见明显背离。若出现过拟合，通常表现为训练损失继续显著下降而验证准确率停滞甚至下跌，本实验并未出现该现象。归功于强数据增强（随机裁剪、亮度抖动等）和 AdamW 权重衰减的正则化作用，模型在较小规模的 MNIST 数据集上依然保持了良好的泛化性能。

收敛性与优化器效果：AdamW 优化器配合学习率指数衰减（初值 0.001，每轮 $\times 0.94$ ）有效地引导参数更新。从日志可以看到，即使后期学习率降至 0.00027，损失和准确率仍在缓慢改善，说明优化器未过早停止学习，最终收敛稳定，不存在震荡或发散现象。

